Experiment 'p_4_4_month_with_highest_temperature_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_4_month_with_highest_temperature_seq
Natural language statement: null
Method signature: p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)

Ensures

- `1 <= hottest <= 12`
- `forall i :: 0 <= i <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]`

Requires

- `|inputs| == 12`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)

The method should respect the following contract:

requires |inputs| == 12, ensures 1 <= hottest <= 12, ensures forall i :: 0 <= i <= |
    ↪ inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLzqHA5hn1ezF7eNRCRyhTNDCCO2
Model: gpt-5.5-2026-04-23
Created at: 1780654998
Finish reason: stop
Usage: 330 tokens in, and 4110 tokens out

```
{
  var maxIdx: int := 0;
  var i: int := 1;

  while i < |inputs|
    invariant |inputs| == 12
    invariant 1 <= i <= |inputs|
    invariant 0 <= maxIdx < i
    invariant forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j]
    decreases |inputs| - i
  {
    if inputs[i] > inputs[maxIdx] {
      var prev := maxIdx;
      assert inputs[i] > inputs[prev];
      assert forall j :: 0 <= j < i ==> inputs[prev] >= inputs[j];

      maxIdx := i;
      assert inputs[maxIdx] >= inputs[i];

      assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j] by {
```

```
      forall j | 0 <= j < i
        ensures inputs[maxIdx] >= inputs[j]
      {
        assert inputs[prev] >= inputs[j];
        assert inputs[maxIdx] == inputs[i];
        assert inputs[i] > inputs[prev];
      }
    }
  } else {
    assert inputs[maxIdx] >= inputs[i];
    assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j];
  }

  assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j];
  assert forall j :: 0 <= j < i + 1 ==> inputs[maxIdx] >= inputs[j] by {
    forall j | 0 <= j < i + 1
      ensures inputs[maxIdx] >= inputs[j]
    {
      if j < i {
        assert inputs[maxIdx] >= inputs[j];
      } else {
        assert j == i;
        assert inputs[maxIdx] >= inputs[i];
      }
    }
  }

  i := i + 1;
 }

 hottest := maxIdx + 1;
 assert i == |inputs|;
 assert 0 <= maxIdx < |inputs|;
 assert |inputs| == 12;
 assert maxIdx < 12;
 assert maxIdx <= 11;
 assert 1 <= hottest <= 12;
 assert hottest - 1 == maxIdx;

 assert forall j :: 0 <= j <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[j] by {
  forall j | 0 <= j <= |inputs| - 1
    ensures inputs[hottest - 1] >= inputs[j]
  {
    assert j < |inputs|;
    assert inputs[maxIdx] >= inputs[j];
    assert hottest - 1 == maxIdx;
  }
 }
}
```

Final Program

```
method p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)
      requires |inputs| == 12
      ensures 1 <= hottest <= 12
      ensures forall i :: 0 <= i <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]
{
  var maxIdx: int := 0;
  var i: int := 1;

  while i < |inputs|
    invariant |inputs| == 12
    invariant 1 <= i <= |inputs|
    invariant 0 <= maxIdx < i
    invariant forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j]
    decreases |inputs| - i
  {
    if inputs[i] > inputs[maxIdx] {
      var prev := maxIdx;
      assert inputs[i] > inputs[prev];
      assert forall j :: 0 <= j < i ==> inputs[prev] >= inputs[j];

      maxIdx := i;
      assert inputs[maxIdx] >= inputs[i];

      assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j] by {
        forall j | 0 <= j < i
          ensures inputs[maxIdx] >= inputs[j]
        {
          assert inputs[prev] >= inputs[j];
          assert inputs[maxIdx] == inputs[i];
          assert inputs[i] > inputs[prev];
        }
      }
    } else {
      assert inputs[maxIdx] >= inputs[i];
      assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j];
    }

    assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j];
    assert forall j :: 0 <= j < i + 1 ==> inputs[maxIdx] >= inputs[j] by {
      forall j | 0 <= j < i + 1
        ensures inputs[maxIdx] >= inputs[j]
      {
        if j < i {
          assert inputs[maxIdx] >= inputs[j];
        } else {
          assert j == i;
          assert inputs[maxIdx] >= inputs[i];
        }
      }
    }
```

```
    i := i + 1;
  }

  hottest := maxIdx + 1;
  assert i == |inputs|;
  assert 0 <= maxIdx < |inputs|;
  assert |inputs| == 12;
  assert maxIdx < 12;
  assert maxIdx <= 11;
  assert 1 <= hottest <= 12;
  assert hottest - 1 == maxIdx;

  assert forall j :: 0 <= j <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[j] by {
    forall j | 0 <= j <= |inputs| - 1
      ensures inputs[hottest - 1] >= inputs[j]
    {
      assert j < |inputs|;
      assert inputs[maxIdx] >= inputs[j];
      assert hottest - 1 == maxIdx;
    }
  }
}
```

Total Token Usage

Input tokens: 330
Output tokens: 4110
Reasoning tokens: 3427
Sum of 'total tokens': 4440

Experiment Timings

Overall Experiment started at 1780654995350, ended at 1780655099881, lasting 104531ms (104.53 seconds)
Iteration #1 started at 1780654995350, ended at 1780655099880, lasting 104530ms (104.53 seconds)